Face Mask Detection using Deep Learning

Machine Learning Lab — Mini Project Report

Oct 15, 2026 · Prepared by @GDSC

1. Aim

To design and implement a real-time system that detects human faces in an image or video stream and classifies each face as **wearing a mask** or **not wearing a mask**, using a deep learning–based computer vision pipeline.

2. Problem Statement

Manual monitoring of mask compliance in public places (hospitals, offices, transport hubs) is slow, inconsistent, and not scalable. There is a need for an automated system that can:

- Locate every human face present in a photo or a live camera feed, regardless of position or number of people.
- Classify each detected face as **Mask** or **No Mask** with a reliable confidence score.
- Run fast enough (near real time) on ordinary hardware (a laptop CPU, no GPU required) so it can be deployed practically at an entry point or checkpoint.

The challenge is twofold: (a) face detection must still work when a large part of the face is covered by a mask, which breaks classical detectors that rely on nose/mouth patterns, and (b) the classifier must be trained with limited data (a few thousand images) while still generalizing well.

3. Objectives

1. Build a two-stage pipeline: face detection followed by mask classification.
2. Use transfer learning to train an accurate classifier with a small dataset and limited compute.
3. Achieve high accuracy (>95%) on a held-out test set.
4. Provide a working real-time demo on a webcam feed and on static images.

4. Tools, Libraries and Environment

Category	Details
Language	Python 3.12
Deep learning framework	TensorFlow / Keras
Computer vision	OpenCV (`cv2`) — camera I/O, image handling, DNN face detector
ML utilities	scikit-learn — train/test split, classification report, confusion matrix
Visualization	Matplotlib — training curves
Face detector (pretrained)	OpenCV SSD with ResNet-10 backbone (`deploy.prototxt` + `res10_300x300_ssd_iter_140000.caffemodel`)
Base CNN (pretrained)	MobileNetV2 (ImageNet weights)
Hardware used	Standard laptop CPU (no GPU required)

5. Dataset

Class	Images
`with_mask`	2,162
`without_mask`	1,930
Total	**4,092**

- Source: open-source *Face-Mask-Detection* dataset (chandrikadeb7, MIT licence) — a mix of real photographs and images with masks composited onto faces.
- Classes are nearly balanced (53% / 47%), so plain accuracy is a fair evaluation metric.
- Split: **80% train (3,273 images) / 20% test (819 images)**, stratified, fixed random seed (42) for reproducibility.
- The test split is never used during training; all reported results come from it.

6. Proposed Solution — System Architecture

A **two-stage pipeline** is used instead of a single end-to-end model:

```
Input frame → [Stage A: Face Detector] → face bounding boxes
           → crop + resize each face to 224×224 → [Stage B: Mask Classifier]
           → P(Mask), P(No Mask) → draw green/red box + confidence label
```

Stage A — Face Detection (SSD + ResNet-10, pretrained, not trained by us). The frame is resized to a 300×300 blob and passed through a Single-Shot-MultiBox-Detector. It outputs candidate face boxes with confidence scores; boxes below a 0.5 confidence threshold are discarded. A DNN-based detector is used instead of classical Haar cascades because Haar cascades rely on the light/dark pattern of the nose and mouth — exactly what a mask hides — while the DNN detector remains robust to masked faces, head turns and poor lighting.

Stage B — Mask Classification (transfer learning on MobileNetV2). Each detected face is cropped, resized to 224×224, and normalized to the [-1, 1] range expected by MobileNetV2. Because only ~3,300 training images are available — far too few to train a deep CNN from scratch — the convolutional base of MobileNetV2 (pretrained on 1.28M ImageNet images) is reused and **frozen**, and only a small new classification head is trained on top of it.

Why two stages instead of one combined model: face detection is already a solved problem (a good pretrained detector exists), so reusing it lets the classifier see only tightly cropped, consistent face images — a much simpler learning problem that needs far less data.

7. Model Architecture (Classification Head)

#	Layer	Output shape	Parameters
1	Input	224×224×3	0
2	Data augmentation (flip, rotate, zoom, shift)	224×224×3	0
3	MobileNetV2 base (**frozen**)	7×7×1280	2,257,984 (frozen)
4	AveragePooling2D (7×7)	1×1×1280	0
5	Flatten	1280	0

#	Layer	Output shape	Parameters
6	Dense(128, ReLU)	128	163,968
7	Dropout(0.5)	128	0
8	Dense(2, softmax)	2	258

Total parameters: 2,422,210 — only **164,226 (6.8%)** are trainable; the rest are frozen pretrained weights. Output is `P(Mask)` and `P(No Mask)` via softmax, summing to 1.

8. Methodology — Training Procedure

Setting	Value
Optimizer	Adam, learning rate = 1e-4
Loss function	Sparse categorical cross-entropy
Batch size	32
Max epochs	20
Early stopping	stop if validation loss does not improve for 4 epochs; best weights restored
Checkpointing	model saved whenever validation accuracy reaches a new best
Data augmentation	random horizontal flip, rotation (±22°), zoom (±15%), shift (±10%) — training only

Steps carried out:

1. Load and label all images from `dataset/with_mask` and `dataset/without_mask`.
2. Decode, resize to 224×224, and build an 80/20 stratified train/test split.
3. Build the model: augmentation layers → frozen MobileNetV2 → new dense head.
4. Train with Adam optimizer, cross-entropy loss, checkpointing and early stopping.
5. Evaluate on the held-out test set; generate a classification report and confusion matrix.
6. Save the trained model (`mask_detector.keras`) and the training curves.

7. Run inference on static images (`detect_image.py`) and on a live webcam feed (`detect_video.py`).

9. Results

9.1 Learning curves (selected epochs)

Epoch	Train Acc	Val Acc	Train Loss	Val Loss
1	83.4%	98.7%	0.3904	0.0979
5	98.1%	99.0%	0.0600	0.0346
10	98.4%	99.0%	0.0438	0.0276
15	99.3%	99.0%	0.0267	0.0262
20	99.4%	99.2%	0.0262	0.0258

Validation accuracy stays at or above training accuracy throughout, and validation loss decreases steadily — indicating the model is **not overfitting** (dropout and augmentation make training intentionally harder than evaluation).

9.2 Final evaluation on 819 held-out test images

Class	Precision	Recall	F1-score	Support
Mask	99.3%	99.1%	99.2%	433
No Mask	99.0%	99.2%	99.1%	386
Overall accuracy			**99.2%**	819

Confusion matrix

	Predicted: Mask	Predicted: No Mask
Actual: Mask	429 (correct)	4 (error)
Actual: No Mask	3 (error)	383 (correct)

Only **7 misclassifications out of 819** test images.

9.3 Inference speed

~81 ms per 640×480 frame on a laptop CPU (single face), i.e. about **12 frames per second** — fast enough for a usable real-time demo without a GPU.

10. Output

The system draws a bounding box around every detected face:

- **Green box**, label `Mask: 99.4%` → person is wearing a mask.
- **Red box**, label `No Mask: 98.7%` → person is not wearing a mask.

This output is produced in two modes:

1. **Static image mode** (`detect_image.py --image photo.jpg --output result.jpg`) — annotates and saves one image, printing one detection line per face.
2. **Live webcam mode** (`detect_video.py`) — continuously captures frames, detects and classifies all faces in a single batched call per frame, and displays the annotated video until `q` is pressed.

Sample annotated outputs from the held-out test set show correctly labelled masked and unmasked faces with confidence scores above 98% in both classes, including cases with multiple faces in the same frame.

11. Conclusion

A two-stage pipeline combining a pretrained face detector with a transfer-learning-based MobileNetV2 classifier successfully detects face-mask compliance with **99.2% test accuracy** and real-time inference speed (~12 FPS) on ordinary CPU hardware. Transfer learning was essential: freezing the pretrained convolutional base let the model learn an accurate 2-class decision from only ~3,300 training images, instead of requiring hundreds of thousands. The project demonstrates a practical, deployable approach to automated mask-compliance monitoring.

12. Limitations and Future Scope

Limitations

- Binary classification only — a mask worn incorrectly (e.g. below the nose) may still be labelled "Mask".
- Part of the training data uses synthetically composited masks, so real-world accuracy is expected to be somewhat lower than the reported test accuracy.
- Very small, side-on or blurred faces may be missed by the face detector itself.
- No frame-to-frame tracking, so labels can flicker slightly in video.

Future scope

- Add a third class: "mask worn incorrectly".
- Fine-tune (unfreeze) the top layers of MobileNetV2 on more diverse real-world photos.
- Export the model to TensorFlow Lite / ONNX for deployment on Raspberry Pi or Android.
- Build a web interface (Streamlit/Flask) with live alerts and a compliance counter.
- Replace the SSD face detector with a more modern one (YuNet, RetinaFace, YOLO-face) to catch smaller/harder faces.

13. References

1. Sandler, M. et al., *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, Google, 2018.
2. OpenCV ResNet-10 SSD face detector (`res10_300x300_ssd_iter_140000.caffemodel`).
3. Dataset: chandrikadeb7/Face-Mask-Detection (GitHub, MIT licence).